

Sec 2. Linux Fundamentals 1-2-3

Linux is an operating system. You may be unaware that Linux is actually very common. A prominent example is Android OS, which uses the Linux kernel. Linux is also used in cars, servers, supercomputers, and many other applications.

Linux is actually a family of operating systems that are all based on the Linux kernel. There are many variants of Linux because it's open source.

While most flavors of Linux have a graphical user interface (GUI), the command line is really where a lot of the action is. Command lines might seem intimidating, but they allow us to perform functions that would actually be more difficult or time consuming than using a GUI.

Windows is actually a great example of the power of the command line; since the GUI tends to change with every new version of Windows, you'd need to know exactly how to navigate through the different menus to achieve your objective. Commands, however, rarely change, so if you know the command you need to use, you can generally execute a task much faster via the command line than you can if you need to navigate through a lot of windows.

echo – will return any text that we provide. For example, the command 'echo hello!' will return 'hello!'. This might seem trivial, but it's used in a lot of code to provide information back to the user – like progress updates.

whoami – will tell us what user we are currently logged in as.

Interacting With the Filesystem - 1:

Command	Full Name
ls	listing
cd	change directory
cat	concatenate
pwd	print working directory

```
'Using "ls" to to list the contents of the current directory'
tryhackme@linux1:~/Documents$ ls
todo.txt
tryhackme@linux1:~/Documents$ cat todo.txt
Here's something important for me to do later!
```

cat: short for "concatenate," is a versatile utility used to read, display, combine, and manipulate file contents directly from the terminal.

cd it is used to navigate to the parent directory of the current working directory.

ls: Used "ls" to let us know what files are available in the "Documents" folder of this machine. In this case, it is called "todo.txt". We have then used cat todo.txt to concatenate/output the contents of this "todo.txt" file, where the contents are "Here's something important for me to do later!"

Pro tip: You can use cat to output the contents of a file within directories without having to navigate to it by using cat and the name of the directory. I.e. cat /home/ubuntu/Documents/todo.txt

```
'Using "pwd" to list the full path of the current directory'
tryhackme@linux1:~/Documents$ pwd
/home/ubuntu/Documents
tryhackme@linux1:~/Documents$
```

pwd: We already know we're in "Documents" thanks to our terminal, but at this point in time, we have no idea where "Documents" is stored so that we can get back to it easily in the future. I have used the "pwd" (print working directory) command to find the full file path of this "Documents" folder. We're helpfully told by Linux that this "Documents" directory is stored at "/home/ubuntu/Documents" on the machine — great to know! Now in the future, if we find ourselves in a different location, we can just use cd /home/ubuntu/Documents to change our working directory to this "Documents" directory.

find – It is used to search for files and directories recursively based on various criteria such as name, type, size, permissions, and modification time

```
'Using "find" to find a file with the name of "passwords.txt"'
```

```
tryhackme@linux1:~$ find -name passwords.txt  
./folder1/passwords.txt  
tryhackme@linux1:~$
```

"Find" has managed to find the file — it turns out it is located in folder1/passwords.txt — sweet. But let's say that we don't know the name of the file, or want to search for every file that has an extension such as ".txt". Find let's us do that too!

grep – It used to search for patterns (strings or regular expressions) within files or input streams, printing the matching lines to standard output.

```
'Using "grep" to find any entries with the IP address of "81.143.211.90" in "access.log"'
```

```
tryhackme@linux1:~$ grep "81.143.211.90" access.log  
81.143.211.90 - - [25/Mar/2021:11:17 + 0000] "GET / HTTP/1.1"  
200 417 "-" "Mozilla/5.0 (Linux; Android 7.0; Moto G(4))"  
tryhackme@linux1:~$
```

"Grep" has searched through this file and has shown us any entries of what we've provided and that is contained within this log file for the IP.

Sometimes, the information we are looking for is spread across multiple files inside a directory. Instead of checking each file individually, we can tell grep to search recursively through all files and subdirectories. To do this, we use the -R (recursive) option. For example, to search for a variable across all files in the current directory and its subfolders, we can run:

```
grep -R "PRETTY_NAME" /etc/  
grep: /etc/sudoers: Permission denied  
/etc/os-release:PRETTY_NAME="Ubuntu"
```

This will: Search every file in the current directory
Search all subdirectories

Show where the PRETTY_NAME appears

Shell Operators:

Linux operators are a fantastic way to power up your knowledge of working with Linux. There are a few important operators that are worth noting. We'll cover the basics and break them down accordingly to bite-sized chunks.

Symbol / Operator	Description
&	This operator allows you to run commands in the background of your terminal.
&&	This operator allows you to combine multiple commands together in one line of your terminal.
>	This operator is a redirector - meaning that we can take the output from a command (such as using cat to output a file) and direct it elsewhere.
>>	This operator does the same function of the  operator but appends the output rather than replacing (meaning nothing is overwritten).

Operator "&": This operator allows us to execute commands in the background. For example, let's say we want to copy a large file. This will obviously take quite a long time and will leave us unable to do anything else until the file successfully copies. The "&" shell operator allows us to execute a command and have it run in the background (such as this file copy) allowing us to do other things!

Operator "&&": This shell operator is a bit misleading in the sense of how familiar is to its partner "&". Unlike the "&" operator, we can use "&&" to make a list of commands to run for example command1 && command2. However, it's worth noting that command2 will only run if command1 was successful.

Operator ">": This operator is what's known as an output redirector. What this essentially means is that we take the output from a command we run and send that output to somewhere else.

'Using the > Operator'

```
tryhackme@linux1:~$ echo hey > welcome
```

```
tryhackme@linux1:~$ cat welcome  
hey
```

Operator ">>" :This operator is also an output redirector like in the previous operator (>) we discussed. However, what makes this operator different is that rather than overwriting any contents within a file, for example, it instead just puts the output at the end. Following on with our previous example where we have the file "welcome" that has the contents of "hey". If were to use echo to add "hello" to the file using the > operator, the file will now only have "hello" and not "hey". The >> operator allows to append the output to the bottom of the file — rather than replacing the contents.

'Using the >> Operator'

```
tryhackme@linux1:~$ echo hello >> welcome
```

```
tryhackme@linux1:~$ cat welcome  
hey  
hello
```

What is SSH & how Does it Work?

Secure Shell or **SSH** for short and is the common means of connecting to and interacting with the command line of a remote Linux machine. Secure Shell or SSH simply is a protocol between devices in an encrypted form. Using cryptography, any input we send in a human-readable format is encrypted for travelling over a network -- where it is then unencrypted once it reaches the remote machine, such as in the diagram below.

```
root@ip-10-10-81-203: ~
File Edit View Search Terminal Help
root@ip-10-10-81-203:~# ssh tryhackme 10.10.224.147
The authenticity of host '10.10.224.147 (10.10.224.147)' can't be established.
ECDSA key fingerprint is SHA256:ebyzpMB6pKqIQVz5xI4HRzbSW6lr9TjBLqy67Cx4QYE.
Are you sure you want to continue connecting (yes/no)? yes
```

Using SSH to Login to Your Linux Machine: The syntax to use SSH is very simple. We only need to provide two things:

1. The IP address of the remote machine
2. Correct credentials to a valid account to login with on the remote machine

`ssh accountname@MACHINE_IP`

ls -a : Using our ls example, ls informs us that there is only one folder named "folder1" as highlighted. Note that the contents in the screenshots below are only examples. After using the -a argument (short for --all), we now suddenly have an output with a few more files and folders such as ".hiddenfolder". Files and folders with "." are hidden files.

```
'Using ls to view hidden folders'
tryhackme@linux2:~$ ls -a
.hiddenfolder folder1
```

--help: option will list the possible options that the command accepts, provide a brief description and example of how to use it.

```
tryhackme@linux2:~$ ls --help
```

```

-a, --all          do not ignore entries starting with .
-A, --almost-all  do not list implied . and ..
--author           with -l, print the author of each file
-b, --escape       print C-style escapes for nongraphic characters
--block-size=SIZE  with -l, scale sizes by SIZE when printing them;
e.g., '--block-size=M'; see SIZE format below
-B, --ignore-backups do not list implied entries ending with ~
-c                with -lt: sort by, and show, ctime
(time of last

```

man: The manual pages are a great source of information for both system commands and applications available on both a Linux machine, which is accessible on the machine itself and online. To access this documentation, we can use the man command and then provide the command we want to read the documentation for.

```

tryhackme@linux2:~$ man ls
ls - list directory contents

```

Interacting With the Filesystem - 2:

Command	Full Name	Purpose
touch	touch	Create file
mkdir	make directory	Create a folder
cp	copy	Copy a file or folder
mv	move	Move a file or folder
rm	remove	Remove a file or folder
file	file	Determine the type of a file

touch, mkdir: providing the name that we want to assign to the directory

```
tryhackme@linux2:~$ touch note
tryhackme@linux2:~$ ls
folder1 note
```

rm: You can simply remove files by using `rm`. However, you need to provide the `-R` switch alongside the name of the directory you wish to remove.

```
tryhackme@linux2:~$ rm -R mydirectory
tryhackme@linux2:~$ ls
folder1
```

cp, mv: Copying and moving files is an important functionality on a Linux machine, command takes two arguments:

1. the name of the existing file
2. the name we wish to assign to the new file when copying

```
tryhackme@linux2:~$ cp note note2
tryhackme@linux2:~$ ls
folder1 note note2
```

file: It is a powerful utility used to determine the type of a file by analyzing its content rather than relying on its extension.

```
tryhackme@linux2:~$ file note
note: ASCII text
```

List the permissions of all files in the directory:

```
tryhackme@linux2:~$ ls -lh
-rw-r--r-- 1 cmnatic cmnatic 0 Feb 19 10:37 file1
-rw-r--r-- 8 cmnatic cmnatic 0 Feb 19 10:37 file2
```

These three columns are very important in determining certain characteristics of a file or folder and whether or not we have access to it. A file or folder can have a

couple of characteristics that determine both what actions are allowed and what user or group has the ability to perform the given action -- such as Read (r value 4), Write (w value 2), Execute (x value 1)

Symbolic	Numeric	Meaning
<code>rwXr-xr-x</code>	755	Owner can do everything, others can read and execute
<code>rw-r--r--</code>	644	Owner can read/write, others can only read
<code>rwX-----</code>	700	Only the owner has access

Why This Matters: Understanding numeric permissions is important because:

Many Linux commands use numeric values (e.g. `chmod 755 file`)

You can quickly identify security risks

You can control who can access sensitive files

For example:

```
chmod 750 system_overview.txt
'This means: Owner: full access
Group: read + execute
Others: no access'
```

The Differences Between Users & Groups:

Switching between users on a Linux install is to the `su` command. Unless you are the root user (or using root permissions through `sudo`), then you are required to know two things to facilitate this transition of user accounts: The user we wish to switch to and The user's password.

```
tryhackme@linux2:~$ su user2
Password:
user2@linux2:/home/tryhackme$
```

Common Directories:

/etc The `etc` folder (short for etcetera) is a commonplace location to store system files that are used by your operating system. For example, the `sudoers` file highlighted in the screenshot below contains a list of the users & groups that have

permission to run sudo or a set of commands as the root user. Also highlighted below are the "**passwd**" and "**shadow**" files. These two files are special for Linux as they show how your system stores the passwords for each user in encrypted formatting called sha512.

```
tryhackme@linux2:/etc$ ls
shadow passwd sudoers sudoers.d
```

/var The **"/var"** directory, with "var" being short for variable data, is one of the main root folders found on a Linux install. This folder stores data that is frequently accessed or written by services or applications running on the system. For example, log files from running services and applications are written here (**/var/log**), or other data that is not necessarily associated with a specific user (i.e., databases and the like).

```
tryhackme@linux2:/var$ ls
backups log opt tmp
```

/root Unlike the **/home** directory, the **/root** folder is actually the home for the "root" system user. There isn't anything more to this folder other than just understanding that this is the home directory for the "root" user. But, it is worth a mention as the logical presumption is that this user would have their data in a directory such as **"/home/root"** by default.

```
root@linux2:~# ls
myfile myfolder passwords.xlsx
```

/tmp This is a unique root directory found on a Linux install. Short for "temporary", the **/tmp** directory is volatile and is used to store data that is only needed to be accessed once or twice. Similar to the memory on your computer, once the computer is restarted, the contents of this folder are cleared out.

```
root@linux2:/tmp# ls
todelete trash.txt rubbish.bin
```

Terminal Text Editors

Nano: To create or edit a file using nano, we simply use `nano filename` -- replacing "filename" with the name of the file you wish to edit. Once we press enter to execute the command, `nano` will launch! Where we can just begin to start entering or modifying our text. You can navigate each line using the "up" and "down" arrow keys or start a new line using the "Enter" key on your keyboard. to exit, we would want to press "**Ctrl**" and "**X**" to exit Nano.

```
tryhackme@linux3:/tmp# nano myfile
GNU nano 4.8
file

Hello TryHackMe
I can write things into "myfile"
```

VIM: VIM is a much more advanced text editor.

Downloading Files (Wget): A pretty fundamental feature of computing is the ability to transfer files. For example, you may want to download a program, a script, or even a picture. This command allows us to download files from the web via HTTP -- as if you were accessing the file in your browser. We simply need to provide the address of the resource that we wish to download. For example, if I wanted to download a file named "myfile.txt" onto my machine, assuming I knew the web address it -- it would look something like this:

```
wget https://assets.tryhackme.com/additional/linux-fundamentals/part3/myfile.txt
```

Transferring Files From Your Host - SCP (SSH): Secure copy, or SCP, is just that - a means of securely copying files. Unlike the regular cp command, this command allows you to transfer files between two computers using the SSH protocol to provide both authentication and encryption. Working on a model of SOURCE and DESTINATION, SCP allows you to:

- Copy files & directories from your current system to a remote system
- Copy files & directories from a remote system to your current system

Provided that we know usernames and passwords for a user on your current system and a user on the remote system. For example, let's copy an example file

from our machine to a remote machine, which I have neatly laid out in the table below:

Variable	Value
The IP address of the remote system	192.168.1.30
User on the remote system	ubuntu
Name of the file on the local system	important.txt
Name that we wish to store the file as on the remote system	transferred.txt

With this information, let's craft our `scp` command (remembering that the format of SCP is just SOURCE and DESTINATION)

```
scp important.txt ubuntu@192.168.1.30:/home/ubuntu/transferred.txt
```

And now let's reverse this and layout the syntax for using `scp` to copy a file from a remote computer that we're not logged into

Variable	Value
IP address of the remote system	192.168.1.30
User on the remote system	ubuntu
Name of the file on the remote system	documents.txt
Name that we wish to store the file as on our system	notes.txt

The command will now look like the following:

```
scp ubuntu@192.168.1.30:/home/ubuntu/documents.txt notes.txt
```

Serving Files From Your Host - WEB Ubuntu machines come pre-packaged with python3. Python helpfully provides a lightweight and easy-to-use module called "HTTPServer". This module turns your computer into a quick and easy web server that you can use to serve your own files, where they can then be downloaded by another computing using commands such as `curl` and `wget`. Python3's "HTTPServer" will serve the files in the directory where you run the command, but

this can be changed by providing options that can be found within the manual pages. Simply, all we need to do is run `python3 -m http.server` in the terminal to start the module!

```
tryhackme@linux3:/webserver# python3 -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

```
tryhackme@mymachine:~# wget http://MACHINE_IP:8000/myfile
```

Now, let's use `wget` to download the file using the MACHINE_IP address and the name of the file. Remember, because the python3 server is running port 8000, you will need to specify this within your wget command. You will need to open a new terminal to use `wget` and leave the one that you have started the Python3 web server in. This is because, once you start the Python3 web server, it will run in that terminal until you cancel it. Downloading a file from our webserver using wget:

```
tryhackme@linux3:/tmp# wget http://MACHINE_IP:8000/file
```

```
2021-05-04 14:26:16 http://127.0.0.1:8000/file
Connecting to http://127.0.0.1:8000... connected.
HTTP request sent, awaiting response... 200 OK
Length: 51095 (50K) [text]
Saving to: 'file'
```

```
file 100%[=====]
>]
49.90K --.-KB/s in 0.04s
```

```
2021-05-04 14:26:16 (1.31 MB/s) - 'file' saved [51095/51095]
```

You will need to run the wget command in another terminal (while keeping the terminal that is running the Python3 server active).

```
tryhackme@linux3: ~  
File Edit View Search Terminal Help  
tryhackme@linux3:~$ python3 -m http.server  
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...  
█
```

Processes 101: Processes are the programs that are running on your machine. They are managed by the kernel, where each process will have an ID associated with it, also known as its PID. The PID increments for the order in which the process starts. I.e. the 60th process will have a PID of 60. We can use the `ps` command to provide a list of the running processes as our user's session and some additional information such as its status code, the session that is running it, how much usage time of the CPU it is using, and the name of the actual program or command that is being executed:

```
top - 22:36:17 up 1 day, 6:32, 0 users, load average: 0.00, 0.00, 0.00  
Tasks: 5 total, 1 running, 4 sleeping, 0 stopped, 0 zombie  
%Cpu(s): 0.0 us, 0.8 sy, 0.0 ni, 99.2 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st  
MiB Mem : 12630.9 total, 12206.5 free, 83.6 used, 340.9 buff/cache  
MiB Swap: 4096.0 total, 4096.0 free, 0.0 used. 12306.1 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	20	0	892	580	516	S	0.0	0.0	0:00.93	init
100	root	20	0	892	84	20	S	0.0	0.0	0:00.00	init
101	root	20	0	892	84	20	S	0.0	0.0	0:00.07	init
102	cmnatic	20	0	10032	4988	3272	S	0.0	0.0	0:00.08	bash
209	cmnatic	20	0	10872	3704	3188	R	0.0	0.0	0:00.00	top

Note how in the screenshot above, the second process `ps` has a PID of 204, and then in the command below it, this is then incremented to 205. To see the

processes run by other users and those that don't run from a session (i.e. system processes), we need to provide **aux** to the **ps** command like so: `ps aux`

```
root      19802  0.9  0.3  8616  3108 pts/1    T   15:37   0:01 /bin/bash ./background.sh
root      20995  0.0  0.0      0      0 ?        I   15:40   0:00 [kworker/u30:1-events_unbound]
ubuntu    21007  0.0  0.0   7228   592 ?        S   15:40   0:00 sleep 1
root      21008  0.0  0.3  10616  3452 pts/1    R+  15:40   0:00 ps aux
root@linux3: /var/opt#
```

Note we can see a total of 5 processes -- note how we now have "root" and "cmnatic" Another very useful command is the top command; top gives you real-time statistics about the processes running on your system instead of a one-time view. These statistics will refresh every 10 seconds, but will also refresh when you use the arrow keys to browse the various rows. Another great command to gain insight into your system is via the top command.

Managing Processes: You can send signals that terminate processes; there are a variety of types of signals that correlate to exactly how "cleanly" the process is dealt with by the kernel. To kill a command, we can use the appropriately named **kill** command and the associated PID that we wish to kill. i.e., to kill PID 1337, we'd use `kill 1337`. Below are some of the signals that we can send to a process when it is killed:

- SIGTERM - Kill the process, but allow it to do some cleanup tasks beforehand
- SIGKILL - Kill the process - doesn't do any cleanup after the fact
- SIGSTOP - Stop/suspend a process

How do Processes Start: The process with an ID of 0 is a process that is started when the system boots. This process is the system's init on Ubuntu, such as systemd, which is used to provide a way of managing a user's processes and sits in between the operating system and the user. For example, once a system boots and it initialises, systemd is one of the first processes that are started. Any program or piece of software that we want to start will start as what's known as a child process of systemd. This means that it is controlled by systemd, but will run as its own process (although sharing the resources from systemd) to make it easier for us to identify and the likes.

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	20	0	101800	11340	8400	S	0.0	1.1	0:11.74	systemd

Getting Processes/Services to Start on Boot: Enter the use of systemctl -- this command allows us to interact with the systemd process/daemon. systemctl is an easy to use command that takes the following formatting: systemctl [option] [service]

For example, to tell apache to start up, we'll use systemctl start apache2. Seems simple enough, right? Same with if we wanted to stop apache, we'd just replace the [option] with stop (instead of start like we provided). We can do five options with systemctl:

Start

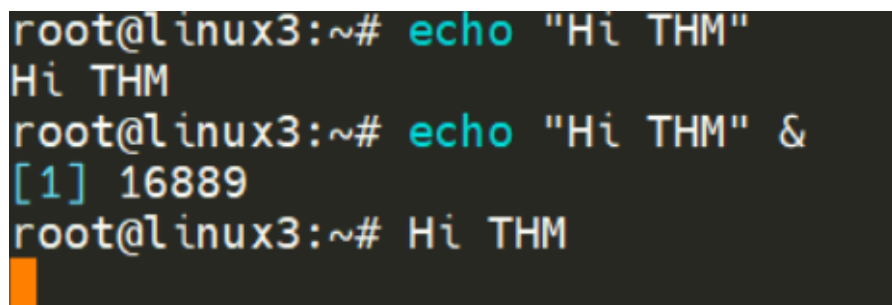
Stop

Enable

Disable

Status

An Introduction to Backgrounding and Foregrounding in Linux: Processes can run in two states: In the background and in the foreground. For example, commands that you run in your terminal such as "echo" or things of that sort will run in the foreground of your terminal as it is the only command provided that hasn't been told to run in the background. "Echo" is a great example as the output of echo will return to you in the foreground, but wouldn't in the background.

A terminal window with a black background and orange text. The prompt is root@linux3:~#. The first command is echo "Hi THM", which outputs Hi THM. The second command is echo "Hi THM" &, which outputs [1] 16889. The third command is Hi THM, which outputs Hi THM.

```
root@linux3:~# echo "Hi THM"
Hi THM
root@linux3:~# echo "Hi THM" &
[1] 16889
root@linux3:~# Hi THM
Hi THM
```

Here we're running `echo "Hi THM"`, where we expect the output to be returned to us like it is at the start. But after adding the `&` operator to the command, we're instead just given the ID of the echo process rather than the actual output -- as it is running in the background. This is great for commands such as copying files because it means that we can run the command in the background and continue on with whatever further commands we wish to execute (without having to wait

for the file copy to finish first). We can do the exact same when executing things like scripts -- rather than relying on the & operator, we can use `Ctrl + Z` on our keyboard to background a process. It is also an effective way of "pausing" the execution of a script or command like in the example below:

```
This will keep on looping until I stop it!
This will keep on looping until I stop it!
This will keep on looping until I stop it!
This will keep on looping until I stop it!
This will keep on looping until I stop it!
This will keep on looping until I stop it!
This will keep on looping until I stop it!
This will keep on looping until I stop it!
This will keep on looping until I stop it!
This will keep on looping until I stop it!
This will keep on looping until I stop it!
This will keep on looping until I stop it!
T^Z
[1]+  Stopped                  ./background.sh
root@linux3:/var/opt#
```

Foregrounding a process: Now that we have a process running in the background, for example, our script "background.sh" which can be confirmed by using the `ps aux` command, we can back-pedal and bring this process back to the foreground to interact with. With our process backgrounded using either `Ctrl + Z` or the `&` operator, we can use `fg` to bring this back to focus like below, where we can see the `fg` command is being used to bring the background process back into use on the terminal, where the output of the script is now returned to us.

```
root      19802  0.9  0.3  8616  3108 pts/1    T   15:37   0:01 /bin/bash ./background.sh
root      20995  0.0  0.0    0      0 ?        I   15:40   0:00 [kworker/u30:1-events_unbound]
ubuntu    21007  0.0  0.0   7228   592 ?        S   15:40   0:00 sleep 1
root      21008  0.0  0.3  10616  3452 pts/1    R+  15:40   0:00 ps aux
root@linux3:/var/opt#
```

9999

Maintaining Your System: Automation: Crontab is one of the processes that is started during boot, which is responsible for facilitating and managing cron jobs. A crontab is simply a special file with formatting that is recognised by the `cron` process to execute each line step-by-step. A cron job is a scheduled task in Unix/Linux systems that runs automatically at specified times or intervals. Crontabs can be edited by using `crontab -e`, where you can select an editor (such as Nano) to edit your crontab. Crontabs require 6 specific values:

Value	Description
MIN	What minute to execute at
HOURL	What hour to execute at
DOM	What day of the month to execute at
MON	What month of the year to execute at
DOW	What day of the week to execute at
CMD	The actual command that will be executed.

Cron Job Generated (you may copy & paste it to your crontab)

```
0 */12 * * * cp -R /home/cmnatic/Documents /var/backups/ >/dev/null 2>&1
```

Your cron job will be run at: (5 times displayed)

- 2021-04-26 00:00:00 UTC
- 2021-04-26 12:00:00 UTC
- 2021-04-27 00:00:00 UTC
- 2021-04-27 12:00:00 UTC
- 2021-04-28 00:00:00 UTC
- ...

Introducing Packages & Software Repos: When developers wish to submit software to the community, they will submit it to an "apt" repository. If approved,

their programs and tools will be released into the wild. Two of the most redeeming features of Linux shine to light here: User accessibility and the merit of open source tools. When using the `ls` command on a Ubuntu 20.04 Linux machine, these files serve as the gateway/registry.

```
ubuntu@ip-10-10-29-121:/etc/apt$ ls
apt.conf.d  auth.conf.d  preferences.d  sources.list  sources.list.d  trusted.gpg.d
ubuntu@ip-10-10-29-121:/etc/apt$
```



```
GNU nano 2.9.3 sources.list

## Uncomment the following two lines to add software from Canonical's
## 'partner' repository.
## This software is not part of Ubuntu, but is offered by Canonical and the
## respective vendors as a service to Ubuntu users.
# deb http://archive.canonical.com/ubuntu bionic partner
# deb-src http://archive.canonical.com/ubuntu bionic partner

deb http://security.ubuntu.com/ubuntu bionic-security main restricted
# deb-src http://security.ubuntu.com/ubuntu bionic-security main restricted
deb http://security.ubuntu.com/ubuntu bionic-security universe
# deb-src http://security.ubuntu.com/ubuntu bionic-security universe
deb http://security.ubuntu.com/ubuntu bionic-security multiverse
# deb-src http://security.ubuntu.com/ubuntu bionic-security multiverse
```

Whilst Operating System vendors will maintain their own repositories, you can also add community repositories to your list! This allows you to extend the capabilities of your OS. Additional repositories can be added by using the `add-apt-repository` command or by listing another provider! For example, some vendors will have a repository that is closer to their geographical location.

Managing Your Repositories (Adding and Removing): Normally we use the `apt` command to install software onto our Ubuntu system. The `apt` command is a part of the package management software also named `apt`. `Apt` contains a whole suite of tools that allows us to manage the packages and sources of our software, and to install or remove software at the same time.

One method of adding repositories is to use the `add-apt-repository` command we illustrated above, but we're going to walk through adding and removing a repository manually. Whilst you can install software through the use of package installers such as `dpkg`, the benefits of `apt` means that whenever we update our system -- the repository that contains the pieces of software that we add also gets checked for updates. In this example, we're going to add the text editor Sublime Text to our Ubuntu machine as a repository as it is not a part of the default Ubuntu repositories. When adding software, the integrity of what we

download is guaranteed by the use of what is called GPG (Gnu Privacy Guard) keys. These keys are essentially a safety check from the developers saying, "here's our software". If the keys do not match up to what your system trusts and what the developers used, then the software will not be downloaded. So, to start, we need to add the GPG key for the developers of Sublime Text 3. (Note that TryHackMe instances do not have internet access and so we're not expecting you to add this to the machine that you deploy, as it would fail.)

1. Let's download the GPG key and use apt-key to trust it:

```
wget -q0 - https://download.sublimetext.com/sublimehq-pub.gpg  
| sudo apt-key add -
```

2. Now that we have added this key to our trusted list, we can now add Sublime Text 3's repository to our apt sources list. A good practice is to have a separate file for every different community/3rd party repository that we add.

2.1. Let's create a file named **sublime-text.list** in **/etc/apt/sources.list.d** and enter the repository information like so:

```
root@linux3:/etc/apt/sources.list.d# touch sublime-text.list  
root@linux3:/etc/apt/sources.list.d# ls  
sublime-text.list  
root@linux3:/etc/apt/sources.list.d#
```

2.2. And now use Nano or a text editor of your choice to add & save the Sublime Text 3 repository into this newly created file:

```
GNU nano 4.8 s  
deb https://download.sublimetext.com/ apt/stable/
```

2.3. After we have added this entry, we need to update apt to recognise this new entry -- this is done using the `apt update` command

2.4. Once successfully updated, we can now proceed to install the software that we have trusted and added to apt using `apt install sublime-text`

Removing packages is as easy as reversing. This process is done by using the `add-apt-repository --remove ppa:PPA_Name/ppa` command or by manually deleting the file that we previously added to. Once removed, we can just use `apt remove [software-name-here]` i.e. `apt remove sublime-text`

Maintaining Your System: Logs: Located in the `/var/log` directory, these files and folders contain logging information for applications and services running on your system. The Operating System (OS) has become pretty good at automatically managing these logs in a process that is known as "rotating". I have highlighted some logs from three services running on a Ubuntu machine:

- An Apache2 web server
- Logs for the fail2ban service, which is used to monitor attempted brute forces, for example
- The UFW service which is used as a firewall

```
ubuntu@ip-172-31-23-158:/var/log$ ls
alternatives.log      dpkg.log              lastlog
alternatives.log.1    dpkg.log.1            letsencrypt
alternatives.log.2.gz dpkg.log.2.gz          lxd
alternatives.log.3.gz dpkg.log.3.gz          mysql
alternatives.log.4.gz dpkg.log.4.gz          syslog
alternatives.log.5.gz dpkg.log.5.gz          syslog.1
alternatives.log.6.gz dpkg.log.6.gz          syslog.2.gz
alternatives.log.7.gz dpkg.log.7.gz          syslog.3.gz
amazon                dpkg.log.8.gz          syslog.4.gz
apache2               dpkg.log.9.gz          syslog.5.gz
appport.log           fail2ban.log           syslog.6.gz
appport.log.1         fail2ban.log.1         syslog.7.gz
apt                  fail2ban.log.2.gz      tallylog
auth.log              fail2ban.log.3.gz      utw.log
auth.log.1            fail2ban.log.4.gz      ufw.log.1
auth.log.2.gz         fontconfig.log         ufw.log.2.gz
auth.log.3.gz         journal                ufw.log.3.gz
auth.log.4.gz         kern.log               ufw.log.4.gz
btmp                  kern.log.1             unattended-upgrades
btmp.1                kern.log.2.gz          wtmp
cloud-init-output.log kern.log.3.gz           wtmp.1
cloud-init.log         kern.log.4.gz
dist-upgrade           landscape
ubuntu@ip-172-31-23-158:/var/log$
```

These services and logs are a great way in monitoring the health of your system and protecting it. Not only that, but the logs for services such as a web server contain information about every single request - allowing developers or administrators to diagnose performance issues or investigate an intruder's activity. For example, the two types of log files below that are of interest:

- access log
- error log

```
ubuntu@ip-172-31-23-158:/var/log/apache2$ ls
access.log      access.log.3.gz  error.log.1      error.log.4.gz
access.log.1    access.log.4.gz  error.log.10.gz  error.log.5.gz
access.log.10.gz access.log.5.gz  error.log.11.gz  error.log.6.gz
access.log.11.gz access.log.6.gz  error.log.12.gz  error.log.7.gz
access.log.12.gz access.log.7.gz  error.log.13.gz  error.log.8.gz
access.log.13.gz access.log.8.gz  error.log.14.gz  error.log.9.gz
access.log.14.gz access.log.9.gz  error.log.2.gz   other_vhosts_access.log
access.log.2.gz  error.log        error.log.3.gz
```

There are, of course, logs that store information about how the OS is running itself and actions that are performed by users, such as authentication attempts.